

Mode B Implementation Plan

Schema-Driven Model Objects — Review, Gap Analysis, and Roadmap

This document reviews the current state of the VistA-Labyrinth codebase from the perspective of Mode B (schema-driven, code-free model objects), identifies what is complete, what is partially implemented, and what is absent, and presents a phased implementation roadmap with concrete deliverables for each phase.

1. What Has Been Built — Current State Review

The following units exist and compile against Delphi 12.2 with the ModelDesigner SQLite database:

VL.Model.Prototype	470 lines	TVLModelPrototype — abstract base class. Persistence state (New/Clean/Dirty/Deleted/Conflict), lifecycle state (Draft/Validating/Valid/Invalid/Active/Archived/Error), GUID + Int64 identity, RowVersion, UTC timestamps, BeginLoad/EndLoad gate, OnStateChanged callback. Fully implemented.
VL.Registry.Core	347 lines	TModelRegistry — identity map with FByGUID (primary) and FByID (secondary) indexes. CreateNew generic factory. Register/Release/AcknowledgeIDAssigned. Enumeration: All, ByType, Dirty. Observer notification. Fully implemented.
VL.Registry.Observer	116 lines	IRegistryObserver interface. TBaseRegistryObserver — non-reference-counted base (TObject, not TInterfacedObject) preventing double-free on shutdown. Fully implemented.
VL.Registry.Debug	391 lines	TRegistrySnapshot (point-in-time capture). TStateTransitionEntry record. TStateTransitionLog — in-memory ordered log, session-scoped, FlushToDatabase stub for Phase 2. TRegistryDiagnostics — text formatting. Substantially implemented; FlushToDatabase is a stub.
VL.Registry.Persistence	83 lines	TRegistryPersistence — stub only. Flush and WriteAuditEntry are no-ops pending Phase 2. Placeholder unit.
VL.Instance.Model	270 lines	TModelObjectInstance — TVLModelPrototype subclass. TypeID, TDictionary property bag, TInstancePropertyDef injection, DoValidate (required-field check), PropertySnapshot, GetPropertyValue/SetPropertyValue. Fully implemented for Mode B core.

VL.Instance.Persistence	577 lines	TInstanceDatabase — SQLite connection, WAL mode, FK pragma, schema V1 auto-migration. TInstanceStore — Store (INSERT/UPDATE in transaction), StoreAll (all dirty), LoadAll, LoadByType, DeleteInstance. Substantially implemented; optimistic concurrency check is absent.
VL.Debug.MockModel	90 lines	TGenericModelObject — synthetic Mode A demo subclass for testing registry mechanics without a real domain backend.
VL.Debug.TypeSelectForm	121 lines + DFM	Modal dialog for type selection when creating a new instance with All Types filter active. Fully implemented.
VL.Debug.MainForm	1338 lines + DFM	Registry Viewer application shell. TActionList-driven UI. Left: Type Definitions tab (browse/edit TObjectTypeRecord), Instances tab (filter + list). Right: Properties tab (type mode), Instance tab (merged envelope + TStringGrid domain), History tab. Substantially implemented; read-only after load (no live domain backend).

2. Gap Analysis — What Is Missing for a Complete Mode B System

Legend: ✓ **Done** ■ **Missing / Stub**

2.1 Registry layer

- ✓ Identity map with GUID + ID indexes
- ✓ Observer pattern — IRegistryObserver, TBaseRegistryObserver
- ✓ CreateNew generic factory
- CreateInstance(TypeID, Defs) — non-generic Mode B factory replacing two-step CreateNew + SetPropertyProperties
- FindByTypeAndProperty(TypeID, PropName, Value) — query within registry by property value (needed for cross-instance lookups)

2.2 Instance model layer

- ✓ TModelObjectInstance with property bag
- ✓ DoValidate — required-field check
- ✓ SetPropertyProperties / PropertySnapshot
- Extended validation: data type format check (integer, date, boolean) using TInstancePropertyDef.DataType
- Cross-property validation hook — abstract method for client to inject business rule validation without subclassing
- Property change event per property (currently only aggregate dirty signal via OnStateChanged)

2.3 Persistence layer

- ✓ TInstanceDatabase — SQLite, WAL, FK, schema V1 auto-migration

- ✓ Store (INSERT/UPDATE in transaction), StoreAll, LoadAll, LoadByType, DeleteInstance
- Optimistic concurrency — RowVersion comparison before UPDATE; PersistenceState=Conflict on mismatch
- Bulk load with type def injection — LoadAll currently leaves PropertyDefs empty; injection must happen after load
- Schema migration V2+ framework — version table exists but no migration runner for future schema changes
- TStateTransitionLog.FlushToDatabase — audit log persistence (FlushToDatabase method exists as stub)
- Soft-delete support — DeleteInstance is a hard DELETE; no archived_at column or soft-delete flag

2.4 Lifecycle layer

- ✓ Bootstrap lifecycle states in TVLModelPrototype enum
- ✓ TStateTransitionLog — in-memory session log
- LifecycleDefinition table — type-specific state names and terminal flags loaded from DB
- LifecycleTransition table — allowed transitions, triggers, guard expressions
- InstanceLifecycle table — current state per instance in DB
- LifecycleHistory table — append-only audit log (event sourcing precursor)
- TLifecycleStore — load state machine, validate transitions, persist history
- Transition guard evaluation — expression evaluator or pluggable delegate for guard conditions

2.5 Validation and external integration layer

- ✓ Local structural validation (DoValidate, required fields)
- Data type format validation (integer, date, boolean, reference)
- RPC Gateway validation adapter — submit property bag, receive structured error set, map to ValidationErrors
- Async validation support — LifecycleState=Validating during external call; Valid/Invalid on response with error mapping
- Anti-Corruption Layer — translate between Delphi string bag and VistA/MUMPS field representation

2.6 Schema / type definition layer

- ✓ TObjectTypeRecord + TObjectPropertyRecord via TObjectModelData
- ✓ Type CRUD in Registry Viewer (add/edit/delete type and properties)
- JSON Schema import/export — load type definitions from external JSON Schema documents
- Type versioning — bump version on structural change, track incompatible property removals
- Reference type resolution — RefTypeID property currently stored but not enforced or resolved at instance creation
- Array property handling — IsArray flag stored but property bag holds single string; no list serialisation

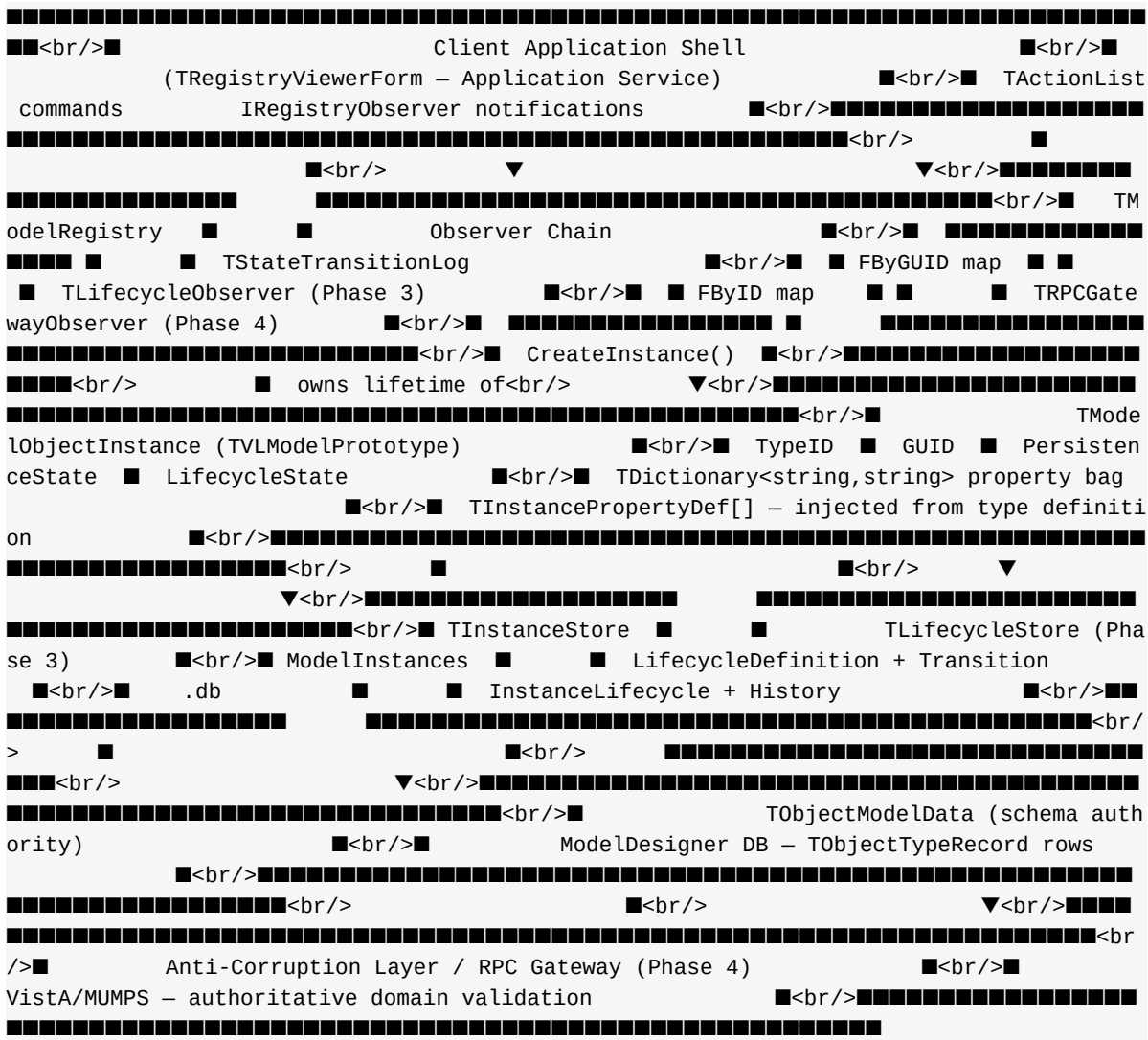
2.7 Application / client shell layer

- ✓ Registry Viewer as Application Service shell
- ✓ TActionList — command/query separation at UI level
- ✓ TTypeSelectForm — type picker dialog
- ✓ TStringGrid — property value editing

- Property editor by data type — date picker, integer spinner, boolean checkbox inline in TStringGrid
- Instance search / filter within registry by property value
- Cross-instance view — show all instances of a type as a grid with properties as columns
- Dirty instance indicator in list (visual badge)
- Unsaved-changes warning on close / DB switch

3. Target Architecture — Mode B Complete System

The following diagram shows the target component structure for a complete Mode B system:



4. Phased Implementation Plan

Each phase produces independently testable, deployable increments. No phase depends on a subsequent phase being complete. The Registry Viewer application is the integration harness throughout all phases.

Phase 1 Foundation Hardening

In Progress

Objective: Close the gaps in the existing units before adding new layers. Everything in this phase is a correction or completion of already-designed components.

1A — Registry: CreateInstance factory

- Add **TModelRegistry.CreateInstance(ATypeID: string; ADefs: TArray): TModelObjectInstance**
 - Replaces the two-step CreateNew + SetTypeProperties
 - Registry becomes the single creation point — no caller needs to know TModelObjectInstance exists

- Unit: VL.Registry.Core — add method to public interface

1B — Persistence: optimistic concurrency

- Before UPDATE in TInstanceStore.Store: SELECT row_version FROM ModelInstance WHERE guid = ?
 - If DB row_version > instance.RowVersion → set PersistenceState=Conflict, raise exception with detail
 - Caller (ActInstSave) catches Conflict, prompts user: overwrite / reload / cancel
 - Unit: VL.Instance.Persistence — Store method

1C — Persistence: type def injection after LoadAll

- After LoadAll / LoadByType: for each loaded instance, call TObjectModelData.LoadProperties(inst.TypeID) and inst.SetTypeProperties(BuildPropertyDefs)
 - Currently instances load from DB with empty PropertyDefs — validation and grid display are broken until first selection
 - TInstanceStore.LoadAll should accept optional TObjectModelData reference, or caller injects defs post-load
 - Units: VL.Instance.Persistence, VL.Debug.MainForm

1D — Persistence: audit log

- Implement TStateTransitionLog.FlushToDatabase
 - Add audit_log table to ModelInstances.db schema V2
 - INSERT (instance_guid, type_name, from_persistence, to_persistence, from_lifecycle, to_lifecycle, ts)
 - Called by TInstStore.Store after each successful flush
 - Units: VL.Registry.Debug, VL.Instance.Persistence

1E — Persistence: soft delete

- Add archived_at TEXT column to ModelInstance
 - MarkForDeletion sets archived_at = now, lifecycle=Archived instead of hard DELETE
 - LoadAll excludes archived_at IS NOT NULL by default; optional flag to include archived
 - Hard DELETE becomes a separate PurgeInstance method
 - Unit: VL.Instance.Persistence

1F — Viewer: dirty indicator and unsaved-changes guard

- Add visual dirty indicator to ListInstances rows (e.g. bold TypeName when PersistenceState=Dirty or New)
- FormCloseQuery: if any instance IsDirty, prompt "Save before closing?"
 - Unit: VL.Debug.MainForm

Phase 2 Extended Validation

Planned

Objective: Make the validation layer substantive enough to catch real data errors before external submission.

2A — Data type format validation

- Extend DoValidate to check property values against DataType:
 - dtInteger: TryStrToInt
 - dtFloat: TryStrToFloat
 - dtDate / dtDateTime: TryStrToDate / TryStrToDateTime
 - dtBoolean: value in ['true','false','1','0','yes','no']
 - dtReference (RefTypeID set): check registry contains a TModelObjectInstance with TypeID=RefTypeID
- Format errors added to ValidationErrors alongside required-field errors
 - Unit: VL.Instance.Model — DoValidate extension

2B — Cross-property validation hook

- Add abstract/virtual **DoValidateCrossProperty: Boolean** method to TVLModelPrototype
 - Default implementation: no-op, returns True
 - Called after field-level checks in DoValidate
 - Client subclasses or injected delegate can check "EndDate >= StartDate", "RefType instance exists", etc.
 - Units: VL.Model.Prototype, VL.Instance.Model

2C — Property editor by data type in TStringGrid

- TStringGrid currently uses plain text editing for all types. Extend with type-aware editing:
 - dtBoolean: OnSelectCell shows TComboBox with True/False overlay
 - dtDate: OnSelectCell shows TDateTimePicker overlay
 - dtInteger / dtFloat: OnSetEditText validates numeric format inline
 - dtReference: OnSelectCell shows type-filtered instance picker dialog
- New unit: VL.Debug.PropertyEditorOverlay — overlay controls created once, shown/hidden per cell
 - No dynamic construction per keystroke — controls exist statically, repositioned on cell selection

2D — Array property handling

- IsArray properties currently store a single string. Implement JSON array serialisation in the property bag:
 - SetPropertyValues for IsArray property accepts JSON array string '["a","b","c"]'
 - Helper: GetPropertyValuesArray / SetPropertyValuesArray returning TArray
 - TStringGrid shows array values comma-separated; clicking opens a simple list editor dialog
 - Units: VL.Instance.Model, VL.Debug.MainForm

Phase 3 Lifecycle Database

Planned

Objective: Replace bootstrap enum-based lifecycle states with a data-driven state machine per type. Add persistent lifecycle history as the event sourcing foundation.

3A — Lifecycle DB schema

```
-- New DB: ModelLifecycle.db (same directory)<br/>LifecycleDefinition (<br/> id
  TEXT PK, -- e.g. 'TPatient:active'<br/> type_id    TEXT NOT NULL,<br/> state_
name    TEXT NOT NULL,<br/> is_initial  INTEGER DEFAULT 0,<br/> is_terminal INTEGER
DEFAULT 0,<br/> sort_order  INTEGER DEFAULT 0,<br/> description TEXT)<br/>Life
cycleTransition (<br/> id          TEXT PK,<br/> type_id    TEXT NOT NULL,<br/
> from_state TEXT NOT NULL,<br/> to_state    TEXT NOT NULL,<br/> trigger_name
  TEXT, -- e.g. 'Submit', 'Approve', 'Archive'<br/> guard_expr TEXT -- fut
ure: expression for guard condition<br/>)<br/>InstanceLifecycle (<br/> instance_guid
TEXT PK FK -> ModelInstance(guid),<br/> current_state TEXT NOT NULL,<br/> entered_at
  TEXT NOT NULL)<br/>LifecycleHistory (<br/> id          INTEGER PK AUTOINC
REMENT,<br/> instance_guid TEXT NOT NULL,<br/> from_state  TEXT,<br/> to_state
  TEXT NOT NULL,<br/> trigger_name TEXT,<br/> actor      TEXT, -- user or
system agent<br/> reason    TEXT,<br/> ts          TEXT NOT NULL DEFAULT (da
tetime('now'))<br/>)
```

3B — TLifecycleStore

- New unit: **VL.Lifecycle.Persistence**
 - LoadDefinitions(ATypeID) — load LifecycleDefinition rows
 - LoadTransitions(ATypeID) — load allowed transitions
 - IsTransitionAllowed(AInstance, ATrigger) — validate trigger

- ApplyTransition(AInstance, ATrigger, AActor, AReason) — write InstanceLifecycle + LifecycleHistory, update TModelObjectInstance.LifecycleState
- GetHistory(AInstanceGUID) — ordered LifecycleHistory for one instance

3C — Viewer: lifecycle tab

- Replace History tab content with two sub-sections:
 - Transition panel: available trigger buttons (from TLifecycleStore.LoadTransitions for current state)
 - History list: LifecycleHistory rows for selected instance (replaces TStateTransitionLog which becomes session-only)
- Lifecycle definition editor in Type Definitions tab:
 - Add/edit states for selected type
 - Add/edit allowed transitions between states

Phase 4 External Validation and RPC Integration

Planned

Objective: Connect the Mode B property bag to the RPC Gateway for external validation and VistA domain operations. Implement the Anti-Corruption Layer.

4A — Validation adapter interface

- New unit: **VL.Validation.Adapter**
- Interface: **IEternalValidator**
 - function ValidateAsync(AInstance: TModelObjectInstance; ACallback: TValidationCallback): Boolean
 - Callback receives: IsValid, TArray (property name, error message, severity)
 - On call: set LifecycleState=Validating; on callback: set Valid/Invalid + populate ValidationErrors

4B — RPC Gateway adapter

- New unit: **VL.Validation.RPCAdapter** implementing IEternalValidator
 - Translates TModelObjectInstance property bag to RPC call parameters
 - Maps VistA error response fields back to TValidationError array
 - Handles RPC timeout: set LifecycleState=Error, populate ValidationErrors with connectivity message
 - Anti-Corruption Layer: property names in Delphi bag ≠ VistA field numbers — mapping table in configuration

4C — Async lifecycle transition

- Extend TVLModelPrototype with ValidateExternal(IEternalValidator) that drives the async Validating → Valid/Invalid transition
- Viewer: Validate button submits to configured validator; spinner while Validating; result shown in envelope panel

Phase 5 Cross-Instance Views and Query Layer

Planned

Objective: Extend the Registry Viewer beyond single-instance inspection to multi-instance analysis — the natural next step once the persistence and lifecycle layers are solid.

5A — Cross-instance grid view

- New right-panel tab: **Type View** (visible when type filter is set)
 - TStringGrid with instances as rows, type properties as columns
 - Columns built dynamically from TObjectModelData.PropertyByIndex
 - Cells editable — SetPropertyValue on cell change, MarkDirty propagates
 - Dirty rows highlighted; Save All button flushes all dirty instances

5B — Property value search

- Search bar above instance list: "property=value" filter
 - Filters ListInstances to instances where GetPropertyValue(key) contains value
 - Registry-side: TModelRegistry.FindByProperty(TypeID, PropName, Value) returning TArray

5C — Export / import

- Export selected instances to JSON:
 - Array of objects: {guid, type_id, properties: {name: value, ...}, lifecycle_state, created_at, modified_at}
- Import from JSON: validate against type definition, create TModelObjectInstance per entry, offer Save All
 - New unit: VL.Instance.Serialisation

Phase 6 Full Event Sourcing (Optional)

Planned

Objective: Promote LifecycleHistory from audit log to the authoritative source of truth for instance state. Only warranted if HIPAA audit requirements demand complete provenance or if replay-based debugging becomes a priority.

- Replace ModelInstance current-state columns with state reconstructed by replaying LifecycleHistory events
- TInstanceStore.LoadAll rebuilds TModelObjectInstance from event stream
- Snapshot table for performance: periodic projection of current state avoids full replay on every load
- Time-travel query: GetStateAt(AGUID, ATimestamp) replays events up to timestamp
- Command sourcing: every ActInstSave becomes a Command record in a Commands table before execution, enabling undo

■ *This phase is architecturally significant but operationally complex. Defer until Phase 3 lifecycle history has been in production long enough to validate the event log completeness.*

5. Unit Delivery Roadmap

VL.Registry.Core	1A	Add CreateInstance()	VL.Instance.Model
VL.Instance.Persistence	1B	Optimistic concurrency in Store	None
VL.Instance.Persistence	1C	Type def injection in LoadAll	Data.ObjectModel
VL.Registry.Debug	1D	FlushToDatabase implementation	VL.Instance.Persistence
VL.Instance.Persistence	1E	Soft delete + PurgeInstance	None
VL.Debug.MainForm	1F	Dirty indicator + close guard	None
VL.Instance.Model	2A	DataType format validation	System.Math
VL.Model.Prototype	2B	DoValidateCrossProperty hook	None
VL.Debug.PropertyEditorOverlay	2C	New — type-aware cell editors	Vcl.Controls, Vcl.ComCtrls
VL.Instance.Model	2D	Array property helpers	System.JSON
VL.Lifecycle.Persistence	3A/B	New — lifecycle DB + TLifecycleStore	FireDAC
VL.Debug.MainForm	3C	Lifecycle tab + transition buttons	VL.Lifecycle.Persistence
VL.Validation.Adapter	4A	New — IExternalValidator interface	None
VL.Validation.RPCAdapter	4B	New — RPC Gateway validator	VL.Validation.Adapter, RPC Gateway
VL.Model.Prototype	4C	ValidateExternal method	VL.Validation.Adapter
VL.Debug.MainForm	5A	Cross-instance Type View grid	VL.Registry.Core
VL.Registry.Core	5B	FindByProperty query	None
VL.Instance.Serialisation	5C	New — JSON export/import	System.JSON

6. Data Flow — New Instance Lifecycle (All Phases)

```
User: New Instance<br/>
■<br/>
▼<br/>ResolveNewInstanceType() -- Phase 1 (complete)
■ TypeID from filter or TTypeSelectForm<br/>
▼<br/>TObjectModelData.LoadProperties(TypeID) → TInstancePropertyDef[]<br/>
■<br/>
▼<br/>TModelRegistry.CreateInstance(TypeID, Defs) -- Phase 1A (gap)<br/>
■ Creates TModelObjectInstance, pre-populates bag, registers
■ PersistenceState=New, LifecycleState=Draft<br/>
▼<br/>User edits properties in TStringGrid<br/>
■ SetPropertyValue → MarkDirty → OnStateChanged<br/>
■ PersistenceState=Dirty<br/>
▼<br/>ActInstValidate (local) -- Phase 1 (c
```

complete)
 ■ DoValidate: required fields checked
 ■ Phase 2A: data type format checked
 ■ LifecycleState: Draft → Validating → Valid/Invalid
 ▼
ActInstValidateExternal -- Phase 4
 ■ IExternalValidator.ValidateAsync()
 ■ LifecycleState=Validating during call
 ■ Anti-Corruption Layer maps bag to RPC parameters
 ■ VistA response maps to ValidationErrors
 ■ LifecycleState → Valid or Invalid
 ▼
ActInstSave -- Phase 1 (complete + 1B concurrency)
 ■ RowVersion check against DB (Phase 1B)
 ■ INSERT/UPDATE ModelInstance + InstanceProperty
 ■ AcknowledgeSave: PersistenceState=Clean, RowVersion++
 ■ LifecycleState: Valid → Active
 ■ Audit log entry written (Phase 1D)
 ▼
TLifecycleStore.ApplyTransition(trigger) -- Phase 3
 ■ Checks LifecycleTransition table for allowed trigger
 ■ Writes InstanceLifecycle + LifecycleHistory
 ■ LifecycleState updated in envelope
 ▼
Instance observable in Registry Viewer - - all phases
 TStateTransitionLog records all state changes
 Cross-instance T type View shows all instances (Phase 5A)

7. Immediate Next Steps — Phase 1 Priority Order

The following items are actionable immediately with no new architectural decisions required. They close gaps in already-designed components:

1	1C: Inject type defs after LoadAll — fixes broken validation and domain grid on startup	VL.Instance.Persistence MainForm	Small
2	1A: CreateInstance() factory — simplifies all instance creation call sites	VL.Registry.Core	Small
3	1B: Optimistic concurrency in Store — prevents silent overwrites in multi-session use	VL.Instance.Persistence	Small
4	1F: Dirty indicator + close guard — prevents accidental data loss	VL.Debug.MainForm	Small
5	1D: FlushToDatabase (audit log) — Phase 3 LifecycleHistory foundation	VL.Registry.Debug Persistence	Medium
6	1E: Soft delete — correct archive semantics for MarkForDeletion	VL.Instance.Persistence	Small

VistA-Labyrinth — Mode B Implementation Plan. Current as of June 2026. Subject to revision as implementation progresses.